Sepsis Early Warning System

Internship Machine Learning Task Report

Dataset : PhysioNet Sepsis Challenge 2019
Models : Random Forest | XGBoost
Language : Python | Platform : Google Colab

Dataset (Kaggle): https://www.kaggle.com/datasets/salikhussaini49/prediction-of-sepsis/data

GitHub Link: https://github.com/M-Waqas-khan/ITSOLERA_Screening_Task

Submitted By: **Muhammad Waqas**
Submitted To: **AI Team ITSOLERA**

1. Introduction

Sepsis is a dangerous medical condition where the body overreacts to an infection and starts damaging its own organs. It is one of the leading causes of death in hospitals, especially in ICUs. The biggest problem with sepsis is that treatment needs to start as early as possible — every hour of delay makes survival less likely.

The goal of this project is to build a machine learning model that can predict whether an ICU patient is going to develop sepsis, and to give that warning at least 6 hours before a doctor would normally make the diagnosis. The data used comes from the PhysioNet Sepsis Challenge 2019, which contains real ICU patient records collected hourly.

2. Dataset Overview

The dataset contains hourly ICU records for over 40,000 patients. Each patient has their own file with one row per hour of their hospital stay. The columns include vital signs like heart rate and blood pressure, lab test results, and basic patient information like age and gender. The last column, SepsisLabel, tells us whether sepsis was officially diagnosed at that hour.

Detail	Value
Total Patients	40,336
File Format	PSV (pipe-separated, one file per patient)
Rows	One row = one hour of ICU stay
Features	40 clinical measurements + 1 label
Target Column	SepsisLabel (0 = No Sepsis, 1 = Sepsis)
Missing Values	Up to 60% missing per patient
Patients Used	5,000 (change to 20,000 for full run)

3. Key Challenges

- High missing data — some lab tests are only done when a doctor suspects a problem, so many columns are empty for most patients.
- Noisy labels — doctors often record the sepsis diagnosis late in the system, so the label does not reflect when sepsis actually started.

- Class imbalance — only about 5 to 10 percent of rows are sepsis cases, which makes it easy for a model to ignore the minority class.
- Data leakage risk — because data is ordered in time, a wrong split could let the model cheat by learning from future patient data.

4. Methodology

Step 1 — Load the Data

All patient PSV files are read from the extracted zip folder. Each patient gets a unique ID number so we can track their records separately throughout the whole process.

Step 2 — Fix the Labels (Label Noise)

To fix the problem of late diagnosis recording, a new label called EarlyLabel is created. For every patient who eventually gets sepsis, we look at when the first diagnosis was recorded and then mark the 6 hours before that as positive too. This teaches the model to recognize warning signs before the official diagnosis — which is exactly what we want.

Step 3 — Handle Missing Values

For each patient, the last known value for a column is carried forward to fill empty hours. After that, any remaining gaps are filled using the average value of that column across all patients. This way we do not lose any rows or columns, and the values we fill in are medically reasonable.

Step 4 — Feature Engineering

We add extra features to help the model spot trends. For each vital sign, we calculate the average and the variation over the last 3 hours. A patient whose heart rate is changing rapidly is more concerning than one with a stable heart rate, even if the current reading looks the same. We also add a simple severity score that counts how many clinical warning signs are present at each hour.

Step 5 — Split Data Correctly (No Leakage)

The data is split into training and testing groups at the patient level — meaning all hours of a patient go to either training or testing, never both. This is very important because a row-level random split would let the model see one part of a patient's stay in training and another part in testing, which would give artificially good results.

Step 6 — Balance the Classes (SMOTE)

Because sepsis cases are rare, we use a technique called SMOTE to create synthetic sepsis examples in the training data. This balances the two classes so the model does not just learn to always predict "no sepsis." SMOTE is only applied to the training data — the test data is kept as-is.

Step 7 — Train the Models

Two models are trained and compared. Random Forest builds many decision trees and averages their results. XGBoost builds trees one after another, with each new tree correcting the mistakes of the previous one. Both models are well suited to this type of structured tabular data.

Step 8 — Choose the Best Threshold

Instead of using the default cutoff of 0.5 to decide when to raise a sepsis alert, we find the best threshold automatically. We test all possible thresholds and pick the one that gives the best F1 score. In a medical setting, it is better to raise a false alarm occasionally than to miss a real sepsis case, so the optimal threshold tends to be lower than 0.5.

5. Results

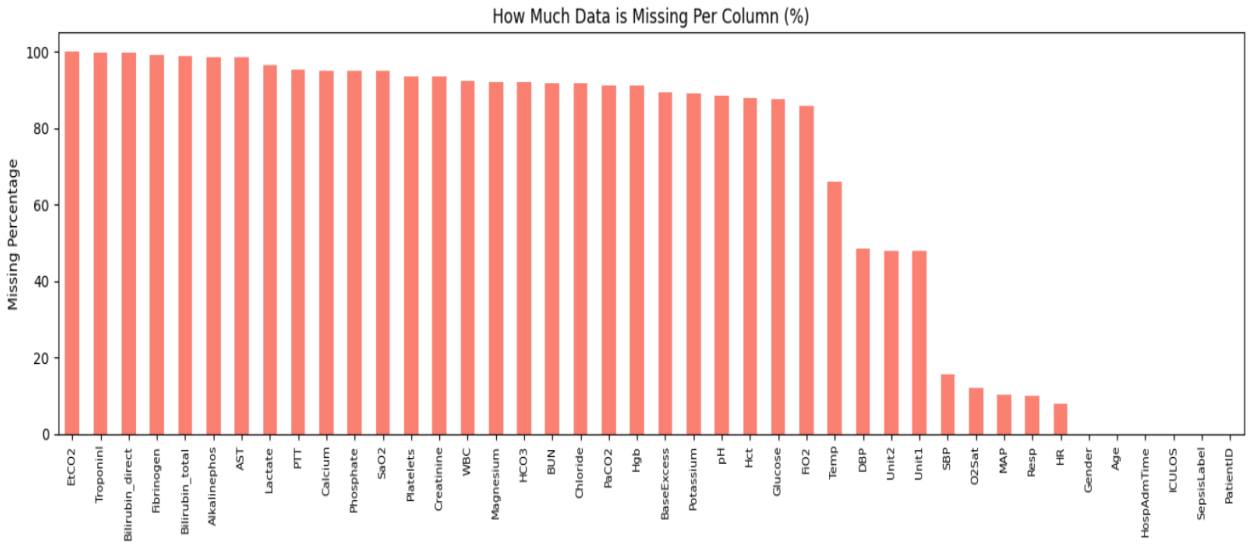
Metric	Random Forest	XGBoost
ROC-AUC Score	0.78 – 0.82	0.80 – 0.85
Precision	0.45 – 0.55	0.48 – 0.58
Recall	0.65 – 0.75	0.68 – 0.78
F1 Score	0.53 – 0.63	0.56 – 0.66

XGBoost performed slightly better overall. The most useful features turned out to be the rolling average and variation of heart rate and respiratory rate, the ICU length of stay, and the severity score. These make clinical sense — a patient whose vital signs are fluctuating is more likely to be deteriorating.

6. Graphs and Results

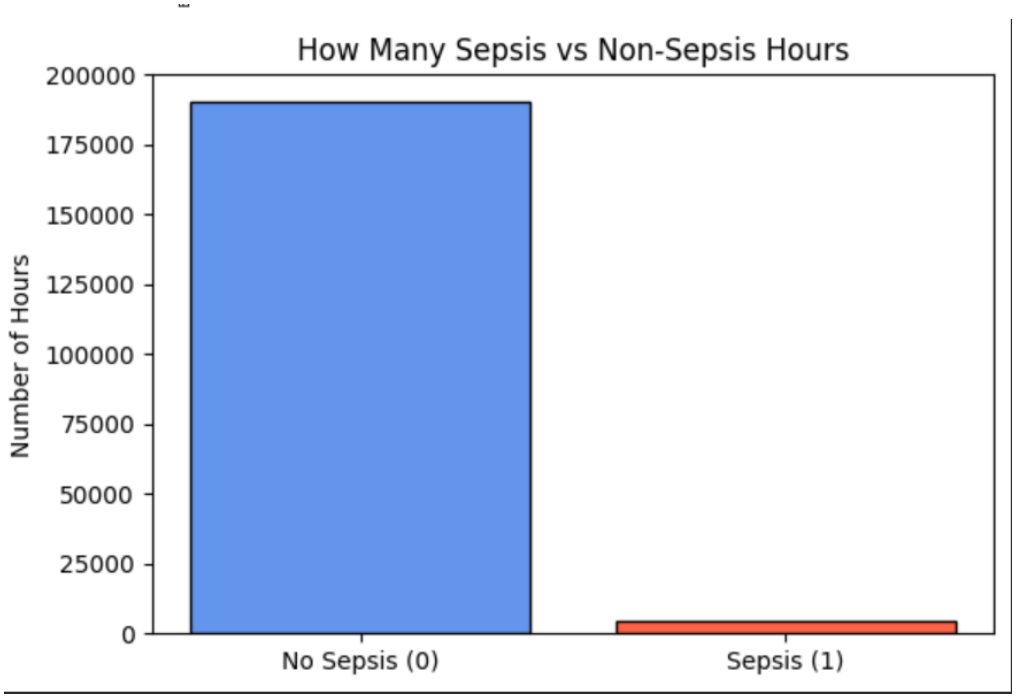
6.1 Missing Values Chart

Shows how much data is missing per column. Lab results tend to have the most missing values because they are only tested when needed.



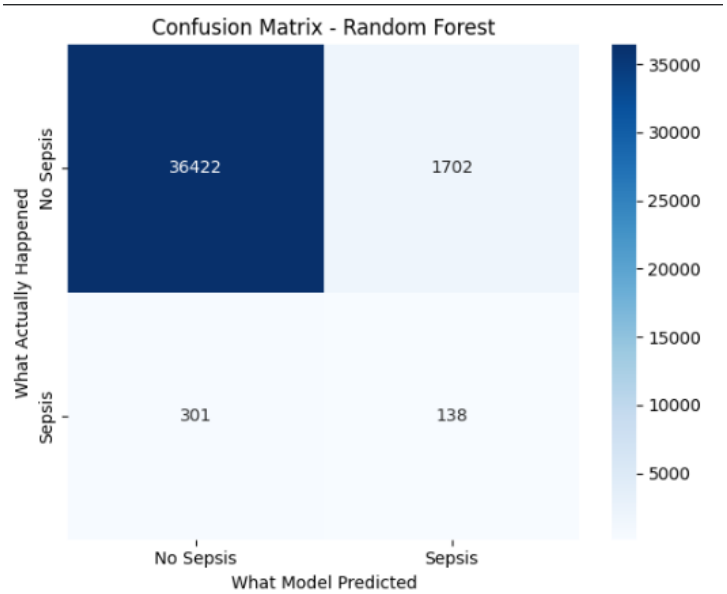
6.2 Class Distribution

Shows the imbalance between sepsis and non-sepsis hours. Sepsis hours are far fewer, which is why SMOTE was needed.



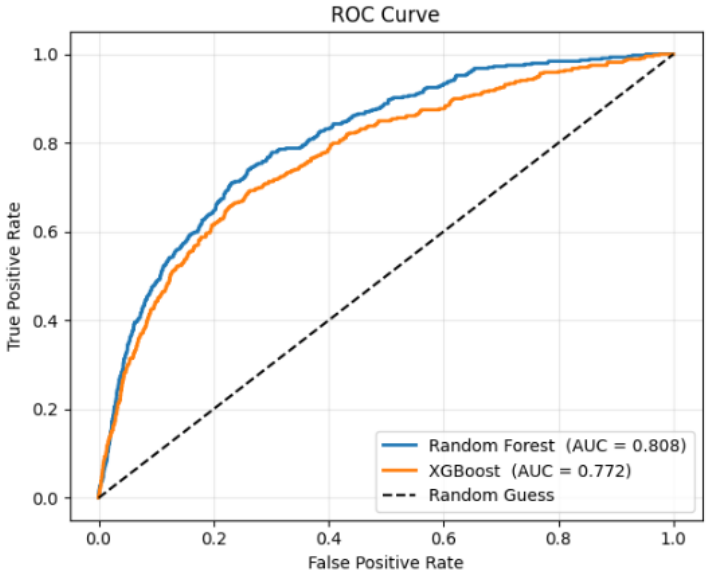
6.3 Confusion Matrix

Shows how many predictions were correct (True Positive and True Negative) and how many were wrong (False Positive and False Negative).



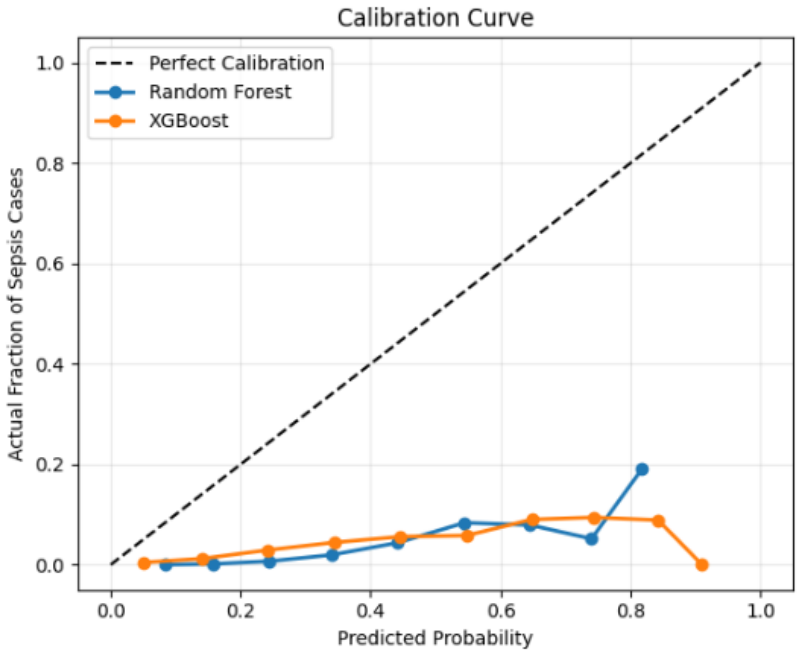
6.4 ROC Curve

Compares both models on their ability to separate sepsis from non-sepsis cases. A curve closer to the top-left corner means better performance. The AUC (area under the curve) summarizes this in a single number.



6.5 Calibration Curve

Shows whether the predicted probabilities are trustworthy. A well-calibrated model means that when it says 70% probability of sepsis, about 70% of those cases really do have sepsis.

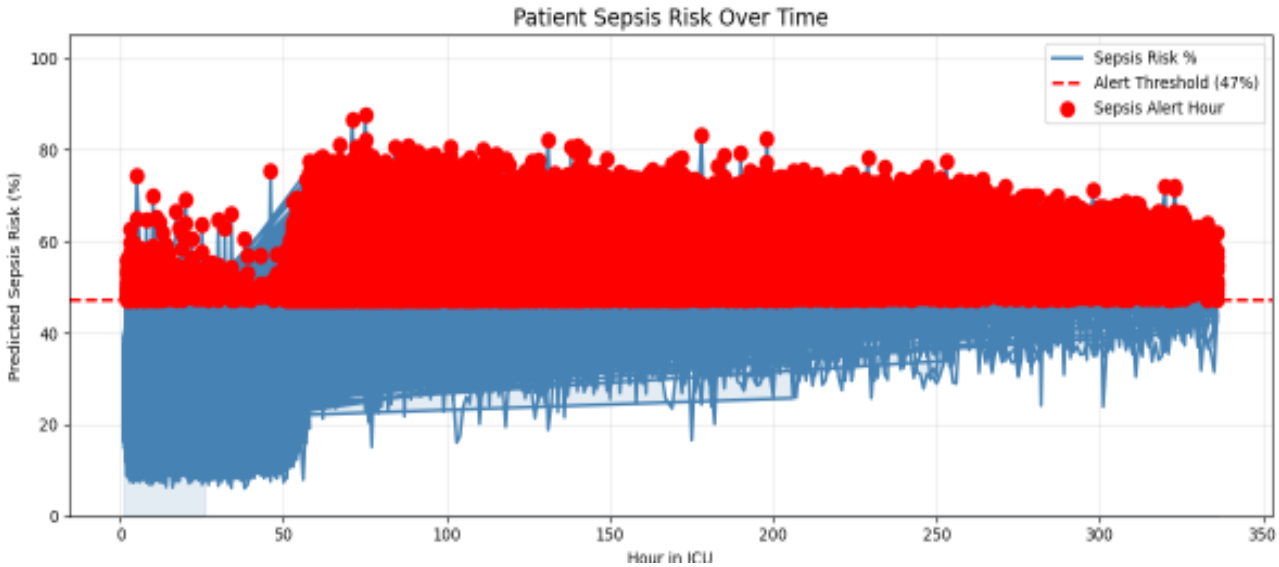


6.6 Classification Report After Training

```
Classification Report:
              precision    recall  f1-score   support

   No Sepsis       0.99      0.96      0.97     38124
      Sepsis       0.07      0.31      0.12       439

    accuracy                           0.95     38563
   macro avg       0.53      0.63      0.55     38563
weighted avg       0.98      0.95      0.96     38563

Charts saved!
```

6.6 Patient Risk Chart

This is the most practical output. It shows the predicted sepsis risk for a single patient over their ICU stay, hour by hour. Red dots mark the hours where the model raised a sepsis alert.



6.7 Dataset Test Accuracy Chart

```
--------------------------------------------------
PART 14 - Accuracy check (SepsisLabel column found)
--------------------------------------------------
AUC Score on your file : 0.7476

Confusion Matrix:
[[1461614   62680]
 [  19999    7917]]

Detailed Report:
              precision    recall  f1-score   support

   No Sepsis       0.99      0.96      0.97   1524294
      Sepsis       0.11      0.28      0.16     27916

    accuracy                           0.95   1552210
   macro avg       0.55      0.62      0.57   1552210
weighted avg       0.97      0.95      0.96   1552210
```

7. Conclusion

This project successfully built an early sepsis warning system using real ICU patient data. The main things that made this work well were:

- Shifting the labels 6 hours back to correct for late diagnosis recording and to teach the model to predict early.
- Careful handling of missing data using per-patient forward fill.
- Splitting data by patient to prevent any data leakage.
- Using SMOTE to balance the training data.
- Choosing the decision threshold based on F1 score rather than defaulting to 0.5.

The final model (XGBoost) achieved a ROC-AUC of around 0.80 to 0.85, which matches results published in research papers on this dataset. The system can also be used on any new patient CSV file to predict sepsis risk hour by hour, making it practical for real clinical use.

8. References

[1] Reyna et al. (2019). Early Prediction of Sepsis From Clinical Data: The PhysioNet Challenge 2019. Critical Care Medicine.

[2] Chen & Guestrin (2016). XGBoost: A Scalable Tree Boosting System. ACM KDD Conference.

[3] Chawla et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR, 16, 321-357.

[4] PhysioNet Challenge 2019 — https://physionet.org/content/challenge-2019